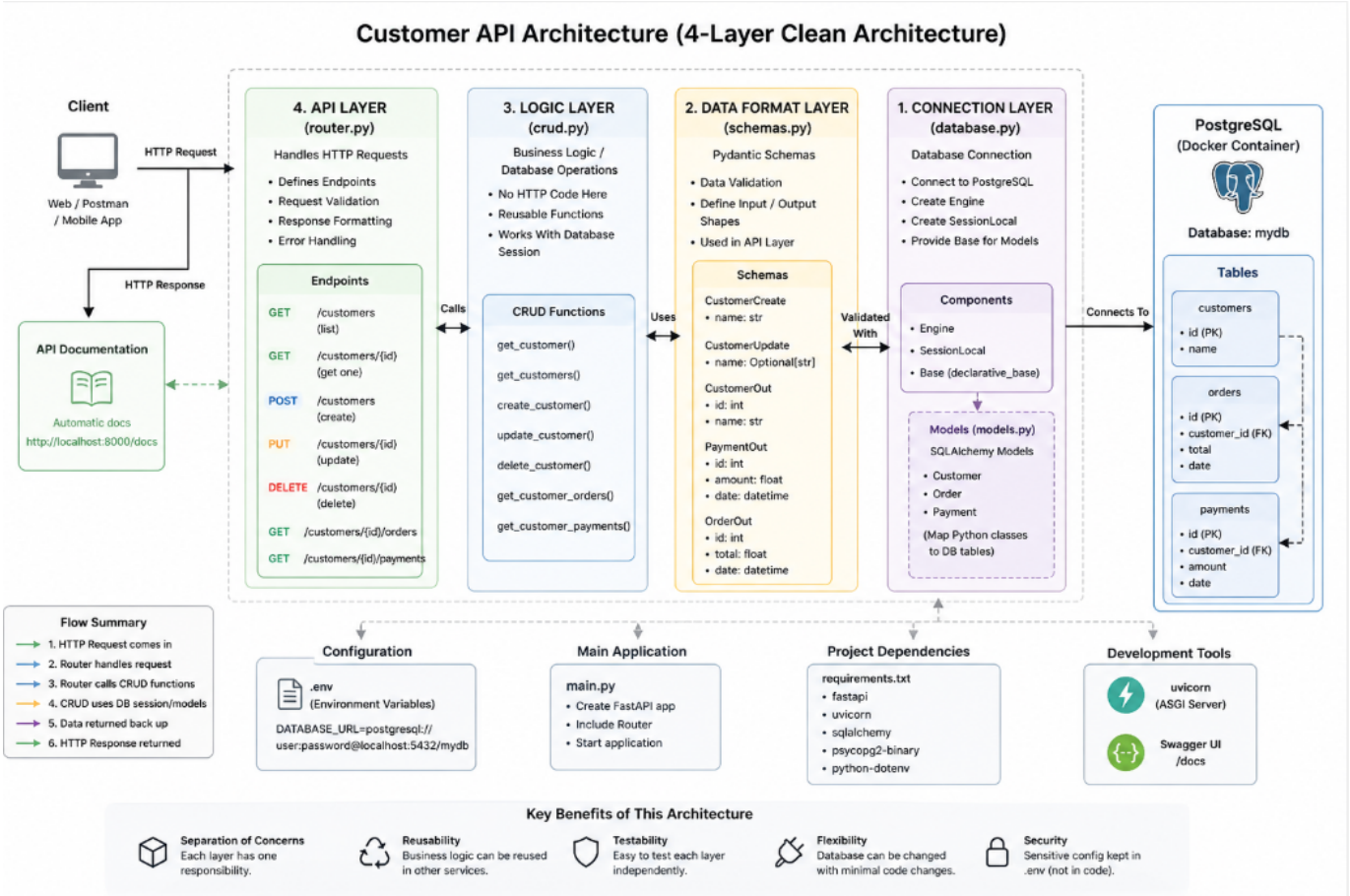
Part 2: Build a Customer API



This project is about building a bridge between your users and your data. Your goal is to create a **Customer API** that allows people to ask for information, and your code will fetch it from the database safely and efficiently.

To keep things professional, we use a **Layered Architecture**. This means we split the code into different files so that each file has only one job.

Step 1: The 4 Layers of Your API

Think of these layers like a restaurant. Each person has a specific role to make sure the meal (your data) is served correctly.

1. database.py (The Connection)

This file acts like the **plumbing**. It sets up the "pipe" between your Python code and the PostgreSQL database.

- **Job:** Open and close the connection.
- **Best Practice:** Use environment variables. Instead of writing your password in the code, you tell Python to look for it in a hidden .env file.

2. schemas.py (The Blueprints)

This file defines what the data looks like. Use **Pydantic** models here.

- **Job:** Validating data. If a user tries to send a "Name" as a number, this layer will stop the error before it hits the database.
- **Example:** You define that a Customer must have an ID (integer) and a Name (string). Use all other fields that the customer table has and validate each column what datatype they belong to so that it only accept those datatypes.

3. crud.py (The Kitchen)

CRUD stands for Create, Read, Update, and Delete, and it represents the core part of the system where most of the data processing takes place.

- The task is to implement database operations using SQLAlchemy (Object-Relational Mapping), writing SQL-like queries in Python.
- This module must strictly interact only with the database and should never communicate with the internet or any external services. Its sole responsibility is to query and manipulate data, such as requesting specific records like “Customer #101” from the database.
- Separate functions should be created for each operation: one for creating data, one for reading data, one for updating data, and one for deleting data.

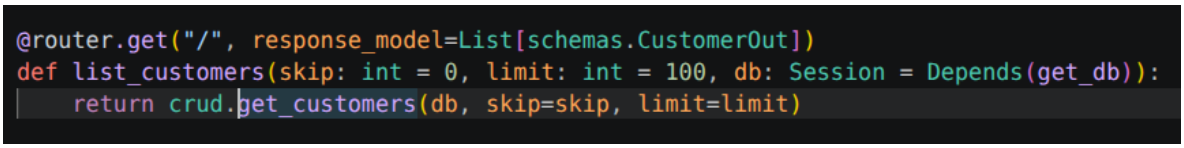
  Example:

```python
# — Customer ——————————————————————————————————————————————
def get_customers(db: Session, skip: int = 0, limit: int = 100):
    return db.query(Customer).offset(skip).limit(limit).all()
```

4. router.py (The Front Desk)

- This is the part of the code that directly interacts with the user. Its responsibility is to handle HTTP requests such as GET and POST.
- When a user accesses an endpoint like `/customers/{customerNumber}`, the router receives the request, invokes the appropriate function from `crud.py`, and returns the response back to the user.
- Different routers should be organized for different functionalities. For example, a GET request can be used to list all customers with pagination limits, a POST request can be used to add or update a customer, and another GET request can retrieve detailed information about a specific customer using their customer number.

  ## **Example:**



Step 2 & 3: How the API Should Behave

Your API needs to be smart. It shouldn't just work; it should handle mistakes gracefully.

The "Get Customer" Logic

When a user asks for a specific customer, your code should follow these steps:

1. Search the database for the ID.
2. **Success:** If found, return the data.
3. **Error Handling:** If the ID doesn't exist, return a **404 Not Found** error. This tells the user "I checked, but it's not there," instead of the whole program crashing.

Pagination (The "List" View)

If you have 10,000 customers, you don't want to show them all on one page.

- **Skip:** How many customers to pass over (like starting at page 2).

- **Limit:** How many customers to show at once (like showing only 10 per page).

Related Data

Customers usually have **Orders** and **Payments**. Your API should be able to show these alongside the customer info. If a customer hasn't bought anything yet, don't break! Just return an empty list: [].

Step 4: Schema Checklist

You need different "blueprints" for different situations:

- **CustomerCreate:** This is for new customers. It might not need an ID yet because the database creates that automatically.
- **CustomerOut:** This is what the user sees. It includes the ID and maybe their order history.
- **CustomerUpdate:** This is for changing info. All the fields here should be optional (maybe the user only wants to change their email, not their name).

Step 5: Logging (The “Activity Tracker”)

Logging is used to record what is happening inside your application. It helps with debugging, monitoring, and understanding system behavior in production.

Why Logging is Important

- Tracks API requests and responses
- Helps debug errors without stopping the system
- Provides visibility into database operations
- Useful for production monitoring and auditing

Where Logging Fits in the Architecture

Logging is not a separate business layer — it is used across all layers:

1. database.py

- Log database connection success or failure
- Log connection closing events

2. schemas.py

- Log validation errors when user input is incorrect

3. crud.py

- Log all database operations:
  - Create operations
  - Read queries
  - Updates
  - Deletes
- Log missing records or failed queries

4. router.py

- Log incoming HTTP requests
- Log responses sent back to users
- Log endpoint errors (like 404 or 500 errors)

Best Practice for Logging

- Use Python's built-in `logging` module
- Set different log levels:
  - `INFO` → normal operations
  - `WARNING` → unexpected but handled situations
  - `ERROR` → failures or exceptions
- Store logs in a file (e.g., `app.log`) for production use
- Avoid printing sensitive data (like passwords or tokens)

Example Use Cases

- "User requested customer ID 101"
- "Database connection established successfully"
- "Customer not found: ID 999"
- "Validation error: invalid email format"

Final Integration Idea

You can also create a **central logging configuration file** (e.g., `logger.py`) and import it into all layers so logging remains consistent across the entire project.

Step 6: Reflection (The "Why")

In professional software, we follow the **Twelve-Factor App** methodology. Here is why we are doing things this way:

Factor II: Dependencies

Question: How do you fix library versions? **Answer:** We use a requirements.txt file and a **Virtual Environment**. This ensures that if you share your code, the other person uses the exact same version of FastAPI as you. It prevents the "It works on my machine" excuse.

Factor IV: Backing Services

Question: How can you change PostgreSQL to another database later? **Answer:** Because we use **SQLAlchemy** (an ORM), our Python code doesn't care which database is behind it. By changing only one line in our configuration, we could switch from PostgreSQL to MySQL or SQLite without rewriting our logic.

Factor III: Config Management

Question: Why keep config in .env? **Answer:** Security and flexibility. You never want to upload your database password to GitHub. By using a .env file, you can have one password for your laptop (development) and a different, stronger password for the real website (production) without changing the code itself.

Simple Summary for Success

1. **Start your database** using Docker.

2. Set up logging (logger configuration) to track system activity across all layers.
3. **Connect** to it in database.py.
4. **Define your data** in schemas.py.
5. **Write your database logic** in crud.py.
6. **Create your endpoints** in router.py.
7. **Test it** at http://localhost:8000/docs to see the automatic documentation!